

Python Fundamentals

Part 3 — Concepts

Strings · Lists · Tuples · Dictionaries · Sets

1. Strings

A string is a sequence of characters enclosed in single quotes (' ') or double quotes (" "). Strings are immutable — once created, they cannot be changed.

```
s = "Hello"
print(s)          # Hello
print(type(s))    # <class 'str'>
```

1.1 String Indexing & Slicing

Each character in a string has an index (starting from 0). Negative indices count from the end.

```
s = "Python"
print(s[0])      # P
print(s[-1])     # n
print(s[0:3])    # Pyt (slicing: start to stop-1)
print(s[::-1])   # nohtyP (reverse the string)
```

Tip — Reverse a string

`s[::-1]` is the shortest way to reverse a string in Python.

1.2 Useful String Operations

Method / Op	Description	Example
<code>len(s)</code>	Length of string	<code>len("hi")</code> → 2
<code>s.lower()</code>	Convert to lowercase	<code>"Hi".lower()</code> → "hi"
<code>s.upper()</code>	Convert to uppercase	<code>"Hi".upper()</code> → "HI"
<code>s.count(x)</code>	Count occurrences of x	<code>"hello".count("l")</code> → 2
<code>s.find(x)</code>	Index of first x (-1 if none)	<code>"hello".find("e")</code> → 1
<code>s.replace(a,b)</code>	Replace a with b	<code>"hi".replace("h","b")</code> → "bi"
<code>s.strip()</code>	Remove leading/trailing spaces	<code>" hi ".strip()</code> → "hi"
<code>s.split()</code>	Split into list of words	<code>"a b".split()</code> → ["a","b"]
<code>' '.join(list)</code>	Join list into string	<code>"-".join(["a","b"])</code> → "a-b"

1.3 Looping Over a String

```
word = "Prime"
for ch in word:
    print(ch)    # P r i m e (one per line)
```

1.4 Palindrome Check

A palindrome reads the same forward and backward (e.g. "madam", "racecar").

Method 1 — Using slicing

```
s = input("enter string: ")
if s == s[::-1]:
    print("Palindrome")
else:
```

```
print("Not a palindrome")
```

Method 2 — Manual reverse with a loop

```
s = input("enter string: ")
rev = ""
for ch in s:
    rev = ch + rev    # build reverse
if s == rev:
    print("Palindrome")
else:
    print("Not a palindrome")
```

1.5 Counting Characters / Spaces

```
s = input("enter string: ")
spaces = 0
for ch in s:
    if ch == ' ':
        spaces += 1
print("Number of spaces:", spaces)
# Or simply:
print(s.count(" "))
```

2. Lists

A list is an ordered, mutable collection of items. Items can be of any type and duplicates are allowed.

```
nums = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True]
print(type(nums))    # <class 'list'>
```

2.1 Creating & Accessing Lists

```
fruits = ["apple", "banana", "kiwi"]
print(fruits[0])      # apple
print(fruits[-1])     # kiwi
print(fruits[0:2])    # ['apple', 'banana']
print(len(fruits))    # 3
```

2.2 Common List Operations

Method / Op	Description	Example
lst.append(x)	Add x at the end	[1,2].append(3) → [1,2,3]
lst.insert(i,x)	Insert x at index i	[1,3].insert(1,2) → [1,2,3]
lst.remove(x)	Remove first x	[1,2,2].remove(2) → [1,2]
lst.pop()	Remove & return last item	[1,2,3].pop() → 3
lst.sort()	Sort in place (ascending)	[3,1,2].sort() → [1,2,3]
sorted(lst)	Return new sorted list	sorted([3,1]) → [1,3]
lst.reverse()	Reverse in place	[1,2,3].reverse() → [3,2,1]
lst.count(x)	Count occurrences of x	[1,2,2].count(2) → 2
x in lst	Membership check	2 in [1,2,3] → True
lst1 + lst2	Concatenate lists	[1]+[2] → [1,2]
lst * n	Repeat list n times	[1]*3 → [1,1,1]

2.3 Looping Over a List

```
nums = [10, 20, 30]
for n in nums:
    print(n)

# With index
for i in range(len(nums)):
    print(i, nums[i])
```

2.4 Average of a List

```
nums = [10, 20, 30, 40]
total = 0
for n in nums:
    total += n
avg = total / len(nums)
print("Average:", avg)    # 25.0

# Or simply:
print(sum(nums) / len(nums))
```

2.5 Merging & Sorting Lists

```
list1 = [1, 2, 7]
list2 = [2, 4, 5]
merged = list1 + list2    # [1, 2, 7, 2, 4, 5]
merged.sort()             # [1, 2, 2, 4, 5, 7]
print(merged)
```

Taking list input from user

```
# Space-separated integers
list1 = list(map(int, input("enter list1: ").split()))
list2 = list(map(int, input("enter list2: ").split()))
result = list1 + list2
result.sort()
print(result)
```

2.6 Finding Duplicates

Print all elements that appear more than once.

```
lst = [1, 2, 3, 2, 4, 3, 5]
seen = set()
duplicates = set()
for x in lst:
    if x in seen:
        duplicates.add(x)
    else:
        seen.add(x)
print(duplicates)    # {2, 3}
```

3. Tuples

A tuple is an ordered, immutable collection. Once created, you cannot add, remove, or change items.

```
t = (1, 2, 3, 4, 5)
print(t[0])    # 1
print(type(t)) # <class 'tuple'>

# Single-element tuple needs a trailing comma
```

```
single = (10,)
```

3.1 Tuple vs List

Feature	List	Tuple
Mutable?	Yes	No
Syntax	[1, 2, 3]	(1, 2, 3)
Speed	Slightly slower	Slightly faster
Use when	Data may change	Data should stay fixed

3.2 Filtering Even & Odd into Tuples

```
t = (1, 2, 3, 4, 5, 6, 7, 8)
evens = tuple(x for x in t if x % 2 == 0)
odds = tuple(x for x in t if x % 2 != 0)
print(evens)    # (2, 4, 6, 8)
print(odds)     # (1, 3, 5, 7)
```

Without comprehension (using loops)

```
t = (1, 2, 3, 4, 5, 6)
even_list = []
odd_list = []
for x in t:
    if x % 2 == 0:
        even_list.append(x)
    else:
        odd_list.append(x)
evens = tuple(even_list)
odds = tuple(odd_list)
print(evens, odds)
```

4. Dictionaries

A dictionary stores data as key–value pairs. Keys must be unique and immutable (strings, numbers, tuples). Values can be anything.

```
student = {"name": "Ali", "age": 20, "marks": 85}
print(student["name"])    # Ali
print(type(student))      # <class 'dict'>
```

4.1 Creating Dictionaries

```
# Empty dict
d = {}
d = dict()

# From pairs
d = {"a": 1, "b": 2}

# From two lists (keys + values)
keys = ["apple", "banana"]
vals = [5, 6]
d = dict(zip(keys, vals))
print(d)    # {"apple": 5, "banana": 6}
```

4.2 Common Dictionary Operations

Operation	Description	Example
-----------	-------------	---------

d[key]	Get value (error if missing)	d["name"]
d.get(key)	Get value (None if missing)	d.get("x")
d[key] = val	Add or update a key	d["age"] = 21
del d[key]	Delete a key	del d["age"]
key in d	Check if key exists	"name" in d
d.keys()	All keys	dict_keys([...])
d.values()	All values	dict_values([...])
d.items()	All (key, value) pairs	dict_items([...])
len(d)	Number of pairs	len(d)

4.3 Looping Over a Dictionary

```
marks = {"Ali": 85, "Sara": 90, "Omar": 78}

for name in marks:           # keys only
    print(name, marks[name])

for name, score in marks.items(): # key + value
    print(name, "→", score)
```

4.4 Word → Length Dictionary

```
words = ["apple", "banana", "kiwi", "cherry", "mango"]
length_dict = {}
for w in words:
    length_dict[w] = len(w)
print(length_dict)
# {"apple": 5, "banana": 6, "kiwi": 4, "cherry": 6, "mango": 5}

# Or with dictionary comprehension:
length_dict = {w: len(w) for w in words}
```

4.5 Menu-Based Student Dictionary

A common pattern is a menu that lets the user choose operations (Add, Update, Search, Display).

```
students = {}

while True:
    print("\nA-Add B-Update C-Search D-Display E-Exit")
    choice = input("Enter choice: ").upper()

    if choice == "A":
        name = input("Name: ")
        marks = int(input("Marks: "))
        students[name] = marks
        print("Added!")

    elif choice == "B":
        name = input("Name to update: ")
        if name in students:
            students[name] = int(input("New marks: "))
            print("Updated!")
        else:
            print("Student not found")

    elif choice == "C":
        name = input("Name to search: ")
```

```

    if name in students:
        print(name, "→", students[name])
    else:
        print("Not found")

    elif choice == "D":
        for name, m in students.items():
            print(name, "→", m)

    elif choice == "E":
        break
    else:
        print("Invalid choice")

```

5. Sets

A set is an unordered collection of unique elements. Duplicates are automatically removed. Sets are mutable and very fast for membership tests.

```

s = {1, 2, 3, 3, 2}
print(s)          # {1, 2, 3} (duplicates gone)
print(type(s))    # <class 'set'>

# Empty set (must use set(), not {})
empty = set()

```

5.1 Common Set Operations

Operation	Description	Example
s.add(x)	Add element	{1}.add(2) → {1,2}
s.remove(x)	Remove x (error if missing)	{1,2}.remove(1)
s.discard(x)	Remove x (no error)	{1}.discard(9)
x in s	Membership test	2 in {1,2} → True
s1 s2	Union	{1,2} {2,3} → {1,2,3}
s1 & s2	Intersection	{1,2}&{2,3} → {2}
s1 - s2	Difference	{1,2}-{2,3} → {1}
s1 ^ s2	Symmetric difference	{1,2}^ {2,3} → {1,3}
len(s)	Number of elements	len({1,2}) → 2

5.2 Checking Common Elements Between Lists

Convert lists to sets, then use intersection.

```

list1 = [1, 2, 3, 4]
list2 = [5, 6, 7, 8]
if set(list1) & set(list2):
    print("Share common elements")
else:
    print("No common elements")

# list1 = [1, 2, 3], list2 = [3, 4] → Share common elements

```

5.3 Unique Characters in a String

```

s = input("enter string: ")
unique = set(s)          # all unique characters
print("Unique chars:", unique)

```

```
print("Count:", len(unique))
```

Note

set(s) treats each character as an element. Spaces and punctuation are also included if present.

6. Quick Comparison

Type	Ordered?	Mutable?	Duplicates?	Syntax
str	Yes	No	Yes	"hello"
list	Yes	Yes	Yes	[1, 2, 3]
tuple	Yes	No	Yes	(1, 2, 3)
dict	Yes*	Yes	Keys unique	{"a": 1}
set	No	Yes	No	{1, 2, 3}

* Dictionaries preserve insertion order (Python 3.7+).

Keep Learning & Keep Exploring!